

Facultad de
Ciencias Exactas,
Ingeniería y Agrimensura



Trabajo práctico II

Aprendizaje automático II

Redes Convolucionales, Detección de Objetos y Redes Recurrentes

Año 2026 - 1° Semestre

Integrantes: Juana Chies Doumecq (C-7554/1), Sebastian Palacio (P-5357/1).

Problema 1

Objetivo

El objetivo de este ejercicio fue diseñar e implementar una red neuronal convolucional entrenada desde cero para la clasificación automática de imágenes de razas de perros. Se utilizó el dataset 70 Dog Breeds Image Dataset (Kaggle), seleccionando un subconjunto de diez clases representativas. El trabajo consistió en desarrollar una arquitectura base y realizar una serie de experimentos sistemáticos sobre la estructura de la red, los hiperparámetros de entrenamiento, las técnicas de regularización y los métodos de aumento de datos. Finalmente, se compararon los resultados obtenidos con el fin de identificar la configuración que proporcionó el mejor desempeño sobre los conjuntos de validación y prueba.

Procedimiento

Dataset e inspección inicial

Se trabajó con un subconjunto de 10 razas del dataset original, seleccionadas con un criterio metodológico dual: incluir razas con rasgos muy distintivos (Chihuahua, Siberian Husky) para facilitar el aprendizaje base, e incorporar razas visualmente similares (Boxer y Bull Mastiff) para evaluar la capacidad del modelo en condiciones de mayor dificultad de clasificación.

Todas las imágenes del dataset presentan una resolución uniforme de 224×224 píxeles, lo que indicó que el conjunto ya había sido curado y estandarizado en origen. El análisis exploratorio visual confirmó alta variabilidad intra-clase en términos de encuadres, poses, fondos e iluminación, lo cual representa un desafío para modelos entrenados desde cero con datos limitados.

Preprocesamiento y Data Augmentation

Dado que el tamaño de entrada estandarizado de 224×224px puede ralentizar el entrenamiento y favorecer el sobreajuste con pocos datos, se decidió redimensionar todas las imágenes a 128×128px.

Los DataLoaders se configuraron con batch size de 32, num_workers=2 y pin_memory=True para optimizar la transferencia a GPU. La normalización basada en estadísticos de ImageNet se aplicó en ambos pipelines, dado que es una práctica estándar que favorece la convergencia incluso en redes entrenadas desde cero.

Arquitecturas implementadas y experimentos

- **E1 — Línea base (CNNBase)**

Arquitectura secuencial de 3 bloques Conv2d(3×3)–BatchNorm2d–ReLU–MaxPool2d, con canales 32→64→128. La cabeza clasificadora emplea AdaptiveAvgPool2d(1,1) para colapsar los mapas de características independientemente de la resolución, seguido de Dropout(0.5) y una capa Linear. Este diseño resulta en aproximadamente 95.000 parámetros entrenables. El optimizador utilizado fue Adam (lr=1e-3, weight_decay=1e-4) con ReduceLROnPlateau (factor=0.5, patience=3) y Early Stopping (patience=10).

- **E2 — Arquitectura profunda (CNNE2)**

Se extendió la red a 4 bloques con filtros hasta 256 canales. En el bloque 3 se empleó un kernel de 5×5 para ampliar el campo receptivo y capturar relaciones espaciales a mayor escala (proporciones corporales, forma del hocico). En el bloque 4 se reemplazó el MaxPool por una convolución con stride=2, permitiendo que la red aprenda qué información preservar durante la reducción dimensional. La cabeza clasificadora incorpora una capa densa oculta adicional de 128 neuronas y dos etapas de Dropout.

- **E3 — Bloques residuales (CNNE3)**

Se implementaron skip connections al estilo ResNet mediante un módulo BloqueResidual con rama principal (Conv–BN–ReLU–Conv–BN) y shortcut (identidad o Conv 1×1 cuando cambian canales o resolución). La arquitectura consiste en un stem inicial seguido de 3 stages con stride=2. Las conexiones residuales garantizan un flujo de gradientes sin interrupciones (evitando el vanishing gradient) y la propiedad de que agregar profundidad nunca empeora el rendimiento base (si $F(x)=0$, la salida es la identidad x).

- **E4 — Regularización mejorada y augmentation agresivo**

Utilizando la misma arquitectura del E3, se modificó el entorno de entrenamiento con tres técnicas adicionales: (1) augmentation agresivo con RandomCrop y RandomGrayscale; (2) label smoothing (label_smoothing=0.1) para penalizar el exceso de confianza en las predicciones; y (3) CosineAnnealingLR como scheduler, reduciendo la tasa de aprendizaje de forma progresiva a lo largo de las 40 épocas.

Decisiones de Diseño

Reducción de resolución de entrada a 128×128 px

Aunque el dataset provee imágenes de 224×224 px, se eligió trabajar a 128×128 px por dos razones prácticas. Primero, reduce el tiempo de cómputo por época de forma significativa. Segundo, con apenas ~1.100 imágenes de entrenamiento, trabajar con una resolución más baja reduce el riesgo de sobreajuste al disminuir la cantidad de detalle innecesario que el modelo podría memorizar.

Detección automática de dispositivo

El dispositivo de entrenamiento se detecta automáticamente verificando la disponibilidad de CUDA y MPS (Apple Silicon), en lugar de hardcodear un dispositivo fijo. Esto garantiza la portabilidad y reproducibilidad del notebook en cualquier entorno sin necesidad de modificar el código.

Desafíos Encontrados

Confusión entre razas morfológicamente similares (molosos)

El Boxer y el Bull Mastiff presentaron sistemáticamente los errores de clasificación cruzada más frecuentes en todos los experimentos. Ambas razas comparten características morfológicas similares: hocico chato, máscara facial oscura y constitución robusta. La red tiene dificultades para extraer los descriptores de grano fino necesarios para distinguirlas sin disponer de un mayor volumen de datos o de pesos preentrenados. Esta limitación se evidencia especialmente en los recalls bajos del Boxer en E1 y E2.

Comportamiento atípico del Chihuahua (sesgo conservador)

En E1 y E2, el modelo mostraba una precisión perfecta pero un recall extremadamente bajo (20–30%) para el Chihuahua: solo predecía esta clase cuando estaba absolutamente seguro, y ante cualquier duda prefería asignar la imagen a otras clases (Pug, Beagle, Golden Retriever). En E3, las skip connections mejoraron el flujo de gradientes y el recall subió al 60%. Sin embargo, en E4, la perturbación visual agresiva eliminó completamente los descriptores de la clase, llevando el recall a 0%.

Techo de rendimiento por escasez de datos

Con aproximadamente 1.100 imágenes de entrenamiento, los modelos alcanzaron un techo del ~72% que no fue posible superar mediante el diseño arquitectónico. El fenómeno de 'manta corta' observado en E3 (mejorar el Chihuahua a costa del Golden Retriever) es la señal

empírica definitiva de que la red ha agotado la variabilidad disponible en el dataset para estabilizar simultáneamente todas las clases.

Conclusiones

Los resultados obtenidos muestran una mejora sostenida entre los experimentos E1, E2 y E3, seguida de una disminución del rendimiento en E4 debido a una regularización excesiva. El incremento en la profundidad y capacidad de la red permitió elevar la exactitud en test del 54% al 70%, mientras que el uso de kernels 5×5 y convoluciones strideadas mejoró significativamente la capacidad de distinguir características espaciales complejas, especialmente en la clase Great Dane.

El modelo E3, basado en bloques residuales, alcanzó el mejor desempeño general con un 72.0% de exactitud en test y una distribución de errores más equilibrada entre las distintas clases. Sin embargo, la mejora obtenida respecto a E2 fue moderada, lo que sugiere que la arquitectura ha alcanzado un límite de rendimiento entrenando desde cero sobre este conjunto de datos.

Por otro lado, E4 evidenció que una regularización demasiado agresiva puede provocar underfitting y deteriorar la capacidad de generalización del modelo, reduciendo la exactitud al 64%. Esto confirma que las estrategias de augmentation y regularización deben ajustarse cuidadosamente al tamaño y características del dataset.

En conclusión, E3 representa la mejor combinación entre capacidad de aprendizaje y generalización. No obstante, los resultados indican que para superar la barrera del 72% de exactitud y reducir las confusiones entre razas visualmente similares, resulta necesario incorporar técnicas de Transfer Learning, aprovechando modelos preentrenados sobre grandes volúmenes de imágenes para extraer representaciones visuales más robustas.

Problema 2

Objetivo

El objetivo fue aplicar transfer learning sobre YOLO11 (Ultralytics) para detectar y clasificar señales de tráfico en imágenes reales. El dataset contiene 15 clases: Green Light, Red Light, Stop y 12 límites de velocidad (10 a 120 km/h). El flujo de trabajo abarcó la inspección del dataset, la configuración del entrenamiento, la comparación de dos variantes del modelo y el análisis exhaustivo de los resultados.

Procedimiento

El dataset ya se encontraba en formato YOLO de Ultralytics con las particiones train, valid y test predefinidas, junto con un archivo data.yaml que centraliza las rutas y la lista de clases. Las estadísticas del conjunto son:

Train: 3.530 imágenes (3 backgrounds, 0 corruptas).

Validación: 801 imágenes (0 backgrounds, 0 corruptas).

Test: 638 imágenes (1 background, 0 corruptas).

Cada archivo de anotación sigue el formato YOLO: una línea por objeto con class_id, x_center, y_center, width y height normalizados entre 0 y 1. Antes de entrenar se graficaron 10 imágenes del conjunto de entrenamiento con sus bounding boxes y etiquetas superpuestas para verificar que las anotaciones eran coherentes con el contenido visual.

Entrenamiento con YOLO11

Se entrenaron dos variantes de YOLO11 a fin de comparar el trade-off entre velocidad de inferencia y precisión: yolo11n (Nano, 2.6M parámetros) y yolo11s (Small, 9.4M parámetros). Ambos modelos fueron inicializados con pesos preentrenados en COCO y sometidos a fine-tuning sobre el dataset de señales de tráfico.

Los hiperparámetros de entrenamiento fueron idénticos para ambas variantes: 50 épocas, imágenes de 640×640 px, batch size de 16, optimizador AdamW (lr=0.000526, momentum=0.9) seleccionado automáticamente por Ultralytics, seed=42 para reproducibilidad y device GPU (CUDA:0, NVIDIA RTX 3060 12 GB). El data augmentation por defecto de Ultralytics (mosaic, randaugment, flips, HSV jitter, erasing) se aplicó durante el entrenamiento y se desactivó en las últimas 10 épocas (close_mosaic=10).

Métricas globales

La tabla siguiente resume las métricas obtenidas sobre el conjunto de test para cada modelo:

Modelo	mAP@50	mAP@50-95	Precisión	Recall
yolo11n	0.955	0.810	0.919	0.930
yolo11s	0.961	0.824	0.929	0.952

Ambos modelos alcanzan un rendimiento muy alto, con mAP@50 superior al 95%. La diferencia es pequeña pero consistente: yolo11s supera a yolo11n en todas las métricas, con una mejora de +0.6 puntos en mAP@50 y +1.4 puntos en mAP@50-95.

Curvas de entrenamiento

Las curvas de box_loss y cls_loss mostraron descenso sostenido en ambos modelos a lo largo de las 50 épocas, sin signos de overfitting en el conjunto de validación. A partir de la época 40 se observa el cierre del mosaic augmentation, lo que produce un descenso abrupto en cls_loss (de ~0.54 a ~0.38 en yolo11n y de ~0.50 a ~0.31 en yolo11s), sin perjuicio de las métricas de validación. Las métricas mAP@50 y mAP@50-95 convergieron gradualmente, alcanzando su pico en las épocas finales del entrenamiento.

Decisiones de Diseño

Elección de YOLO11 y comparación de variantes Nano vs Small

Se eligió YOLO11 de Ultralytics por su arquitectura moderna (bloques C3k2 y módulo C2PSA), su API unificada de entrenamiento y evaluación, y su soporte nativo para transfer learning desde pesos preentrenados en COCO. La comparación de dos variantes fue un requisito explícito del enunciado: permite analizar el trade-off entre velocidad de inferencia y precisión en función de los recursos disponibles en el entorno de despliegue.

Detección automática de dispositivo

El dispositivo de entrenamiento se detecta automáticamente con torch.cuda.is_available() en lugar de hardcodear device='0'. Esto garantiza que el notebook sea ejecutable en cualquier entorno (con o sin GPU) sin necesidad de modificar el código, lo que es relevante para la reproducibilidad y portabilidad del trabajo.

Desafíos Encontrados

Confusión entre semáforos y señales de velocidad: Green Light y Red Light resultaron las clases más difíciles de detectar con precisión. A diferencia de las señales de velocidad, que tienen forma circular estandarizada con texto claro, los semáforos presentan mayor variabilidad en tamaño aparente según la distancia, oclusiones parciales por postes u otros objetos, y variaciones de intensidad lumínica que alteran su apariencia. Esto se refleja en los mAP@50 más bajos de todas las clases para ambos modelos.

Desbalance de instancias por clase: La clase Speed Limit 10 cuenta con solo 3 instancias en el conjunto de test, frente a las ~80 de Stop o Speed Limit 30. Este desbalance hace que las métricas de Speed Limit 10 sean estadísticamente poco confiables y no comparables con las de las demás clases. Para un dataset de producción, se recomendaría recolectar más imágenes de las clases subrepresentadas o aplicar augmentation específico por clase.

Conclusiones

En este ejercicio se demostró que el transfer learning sobre un detector moderno como YOLO11 permite alcanzar un rendimiento muy alto ($mAP@50 > 0.95$) en detección de señales de tráfico con un dataset de tamaño moderado y un entrenamiento relativamente corto (~50 minutos por modelo en GPU). La tabla siguiente sintetiza el trade-off entre ambas variantes:

Modelo	Paráms.	$mAP@50$	$mAP@50-95$	Recall	Entrenamiento	Inferencia
yolo11n	2.6M	0.955	0.810	0.930	48.5 min	8.4 ms/img
yolo11s	9.4M	0.961	0.824	0.952	51.8 min	11.9 ms/img

La diferencia de rendimiento entre yolo11n y yolo11s es pequeña pero consistente: yolo11s supera al nano en todas las métricas, con una mejora de +0.6 puntos en $mAP@50$ y +1.4 puntos en $mAP@50-95$. Sin embargo, este incremento tiene un costo: aproximadamente 3.6 veces más parámetros y un 42% más de tiempo de inferencia (11.9 ms vs 8.4 ms por imagen), con solo 3.3 minutos adicionales de entrenamiento.

Para un escenario con restricciones de latencia (sistema embebido en un vehículo), yolo11n es la mejor opción dado que su precisión es prácticamente equivalente y su inferencia es significativamente más rápida. Para un sistema offline o sin restricciones temporales, yolo11s es preferible por su mayor recall (0.952 vs 0.930), que implica menos señales no detectadas en condiciones reales.

El análisis por clase reveló que los semáforos (Green Light y Red Light) son consistentemente las clases más difíciles, debido a su variabilidad visual intrínseca. Las señales de velocidad, en cambio, obtienen $mAP@50$ superior a 0.97 en ambos modelos, beneficiándose de su diseño estandarizado. Como mejora futura, se podría explorar augmentation específico para semáforos, el uso de modelos de mayor capacidad (yolo11m o yolo11l) o técnicas de focal loss para clases desbalanceadas.

Problema 3

Objetivo

El objetivo de este ejercicio fue diseñar e implementar una arquitectura CRNN (*Convolutional Recurrent Neural Network*) para la clasificación automática de comandos de voz. Para ello se utilizó el dataset **Google Speech Commands v2**, compuesto por grabaciones de audio de corta duración correspondientes a diferentes comandos pronunciados por múltiples hablantes. El proceso incluyó el preprocesamiento de las señales mediante la generación de espectrogramas de Mel, la segmentación temporal de los datos y el entrenamiento de una arquitectura híbrida que combina redes convolucionales para la extracción de características acústicas y redes recurrentes para el modelado de dependencias temporales. Asimismo, se realizaron diversos experimentos sobre la configuración de la CNN, la RNN y los parámetros de segmentación, con el propósito de identificar la combinación que ofreciera el mejor desempeño en la tarea de reconocimiento de voz.

Procedimiento

Comenzamos el flujo de trabajo con la carga del dataset Google Speech Commands V2, del cual se seleccionaron 10 clases específicas de comandos de voz: "yes", "no", "up", "down", "left", "right", "on", "off", "stop" y "go". Los audios se particionaron siguiendo las listas oficiales del corpus, resultando en un conjunto de entrenamiento de 30.769 muestras, un conjunto de validación de 3.703 muestras y un conjunto de evaluación (test) de 4.074 muestras.

A continuación, las señales de audio en formato .wav pasaron por un proceso de extracción de características utilizando la librería librosa, transformándolas en espectrogramas de Mel. Se aplicó una normalización temporal para asegurar que todos los tensores de entrada tuvieran una longitud estandarizada, utilizando técnicas de padding con ceros o truncamiento. Seguidamente, se aplicó una segmentación mediante una ventana temporal deslizante para generar secuencias. Por último, se entrenó la arquitectura híbrida CRNN, implementando una red recurrente (bidireccional) seguida de un clasificador denso ("many-to-one").

Decisiones de Diseño

- **Parámetros acústicos estandarizados:** Se fijó la frecuencia de muestreo en 16 kHz para respetar la naturaleza original del dataset y se estandarizó la longitud de los espectrogramas a un segundo temporal (32 frames).
- **Normalización global robusta:** La media (-57.0897 dB) y la desviación estándar (19.8210 dB) utilizadas para normalizar las señales se calcularon de forma estricta sobre una porción representativa del conjunto de entrenamiento. Esto garantizó que no hubiese fuga de información (*data leakage*) hacia los conjuntos de validación o prueba.
- **Aceleración mediante Caché:** Para evitar la latencia generada por el cálculo de las transformadas de audio en cada época, se decidió precomputar todos los espectrogramas de Mel y almacenarlos directamente en la memoria RAM.

- **Ventana deslizante con solapamiento:** La segmentación temporal de los espectrogramas se realizó aplicando un 50% de solapamiento (*window_size=16*, *hop=8*). Esto permite incrementar los tokens secuenciales y asegura que las transiciones fonéticas fundamentales no sean fraccionadas de forma abrupta entre dos segmentos adyacentes.
- **Agregación "Mean Pooling" en la RNN:** En lugar de capturar únicamente el estado oculto final, se extrajo la media global (*mean pooling*) de los estados de la red recurrente, demostrando ser una decisión clave frente al modelado de audios que terminan en silencios prolongados.

Desafíos Encontrados

- **Procesamiento computacional:** La conversión iterativa desde el formato .wav sin procesar generaba cuellos de botella severos, desafío que se mitigó precomputando y vectorizando los tensores previamente en memoria RAM
- **Dificultad perceptual en comandos fonéticamente similares:** El modelo experimentó confusiones frecuentes al momento de discriminar ciertos comandos que comparten estructura silábica o la misma vocal principal, reflejando dificultades perceptuales que también sufren los seres humanos en entornos acústicos adversos.

Conclusiones

- La transformación de los comandos a espectrogramas de Mel comprobó ser una representación sumamente eficaz para capturar e interpretar los patrones acústicos subyacentes del habla humana.
- La elección de una red híbrida CRNN es altamente favorable, dado que unifica la sólida capacidad de extracción de características locales de las redes convolucionales (CNN) con la eficacia del modelado secuencial y de dependencias a lo largo del tiempo provista por las redes recurrentes (RNN).
- El diseño de solapamiento en la ventana deslizante es determinante para el correcto rendimiento del modelo, ya que asegura que la información de cada transición fonética sea procesada en más de un frame de la secuencia.
- La técnica de "mean pooling" sobre la salida secuencial resultó ser considerablemente más resiliente ante ruidos de fondo y silencios de los comandos de audio.